

**ISTANBUL TECHNICAL UNIVERSITY
COMPUTER ENGINEERING DEPARTMENT**

**BLG22E
COMPUTER ORGANIZATION PROJECT
PROJECT REPORT**

PROJECT NO: 1

GROUP MEMBERS:

150220937 : Amr Nael Zuhdi Taweel

150220116 : Furkan Yalçın

SPRING 2025

Contents

1	INTRODUCTION	1
2	MODULE IMPLEMENTATIONS	1
2.1	Part 1 — 16-bit General-Purpose Register	1
2.2	Part 2 — 32-bit General-Purpose Register	2
2.3	2.2 — Instruction Register (IR)	3
2.4	2.3 — Data Register (DR)	4
2.5	2.4 — General and Scratch Register File (RF)	5
2.6	2.5 — Address Register File (ARF)	5
2.7	Part 3: Arithmetic Logic Unit	7
2.8	Part 4: Arithmetic Logic Unit System	9
3	RESULTS	10
4	DISCUSSION	12
5	CONCLUSION	17

1 INTRODUCTION

The aim of this assignment was to design, code, and simulate a minimal set of storage elements and an Arithmetic Logic Unit (ALU) that together demonstrate the core update mechanisms of a simple CPU.

The assignment comprised four incremental stages:

1. **Part 1:** Created self-contained 16-bit and 32-bit registers offering basic arithmetic and load/clear behaviour.
2. **Part 2:** Assembled those cells into an Instruction Register, Data Register, an 8-entry general/scratch Register File, and a three-entry Address Register File.
3. **Part 3:** Implemented a 32-bit ALU capable of 32 distinct operations plus condition-flag logic.
4. **Part 4:** Integrated everything into a top-level “mini-CPU” connected through multiplexers and a block RAM model.

Our modules passed the supplied test cases, and we further tested our implementations by creating our own simulations.

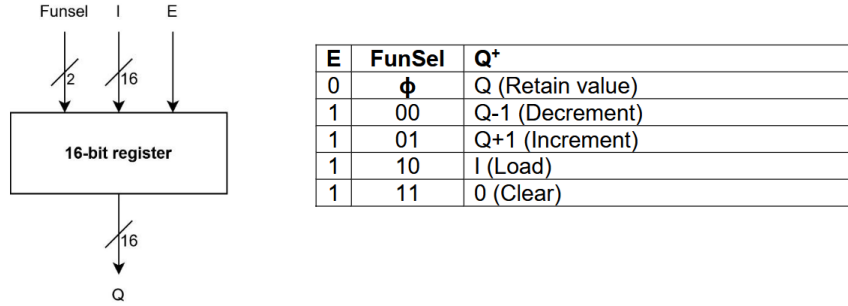
Task distribution:

- **Part 1 & Part 4** (16-/32-bit registers and system integration):
Developed by Amr Nael Zuhdi Taweel, he ran all provided tests and debugged the codes.
- **Part 2 & Part 3** (Register files and ALU):
Designed by Furkan Yalçın, he developed additional test-bench files to check corner-case scenarios.
- **Report:** Co-authored and assembled by Amr Nael Zuhdi Taweel and Furkan Yalçın.

2 MODULE IMPLEMENTATIONS

2.1 Part 1 — 16-bit General-Purpose Register

We designed a 16-bit storage element able to *hold*, *increment*, *decrement*, *load*, or *clear* its value. The same module is later reused for PC, SP, and AR.



The `always @(posedge Clock)` block guarantees that all updates finish together at the next clock edge. Inside, the `if (E)` test decides whether any change is allowed this cycle. A `case(FunSel)` then performs exactly one of the four actions.

```

module Register16bit(I, E, FunSel, Q, Clock
);
    input wire[15:0] I;
    input wire Clock;
    input wire E;
    input wire[1:0] FunSel;
    output reg[15:0] Q;
    always@(posedge Clock)
    begin
        if(E) begin
            case(FunSel)
                2'b00 : Q <= Q - 1'b1;
                2'b01 : Q <= Q + 1'b1;
                2'b10 : Q <= I;
                2'b11 : Q <= 16'b0;

            endcase
        end
    end
endmodule

```

Figure 1: *16bitRegister.v*

We tested our code with simulations and confirmed that our implementation works correctly.

2.2 Part 2 — 32-bit General-Purpose Register

This 32-bit cell retains the exact same control interface as our `Register16bit`: an `E` enable plus a small `FunSel`-encoded `case` on each rising clock edge. All four basic operations (decrement, increment, load, clear) behave identically, merely widened to 32 bits, and the extra modes (byte/half-word write, shift+insert, sign-extend) build directly on the same vector-slice and concatenation techniques we used in the 16-bit design.

```

module Register32bit(FunSel, I, E, Q, Clock);
input wire [2:0] FunSel;
input wire [31:0] I;
input wire E;
input wire Clock;
output reg [31:0] Q;

always@(posedge Clock)
begin
if(E)
begin
case(FunSel)
3'b000 : Q <= Q - 1;
3'b001 : Q <= Q + 1;
3'b010 : Q <= I;
3'b011 : Q <= 32'b0;
3'b100 : begin
Q[31:8] <= 24'b0;
Q[7:0] <= I[7:0];
end
3'b101 : begin
Q[31:16] <= 16'b0;
Q[15:0] <= I[15:0];
end
3'b110 : begin
Q[31:8] <= Q[23:0];
Q[7:0] <= I[7:0];
end
3'b111 : begin
Q[31:16] <= {16{I[15]}};
Q[15:0] <= I[15:0];
end
endcase
end
end
endmodule

```

Figure 2: *32bitRegister.png*

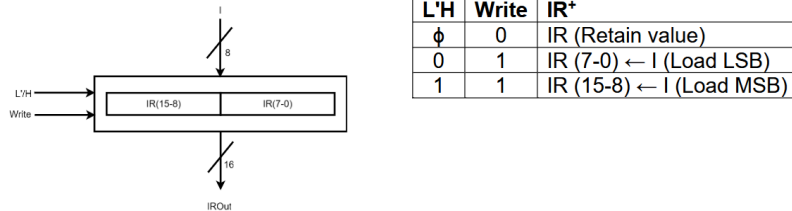
2.3 2.2 — Instruction Register (IR)

The Instruction Register holds the **current** 16-bit opcode so the control logic can decode it on the following cycle.

Because the external memory bus is only eight bits wide, the register is filled in two steps:

- LH=0 and W=1 $\Rightarrow$ low byte is written.
- LH=1 and W=1 $\Rightarrow$ high byte is written.

If W=0 the instruction simply stays unchanged.



```

module InstructionRegister(LH, W, I, clk, IROut);
    input LH;
    input W;
    input [7:0] I;
    input clk;
    output reg [15:0] IROut;

    always@(posedge clk)
    begin
        if(W)
        begin
            case(LH)
                1'b0 : IROut <= {IROut[15:8], I[7:0]};
                1'b1 : IROut <= {I[7:0], IROut[7:0]};
            endcase
        end
    end
endmodule

```

The concatenation operator (`{ }`) splices the new byte with the untouched half already stored.

2.4 2.3 — Data Register (DR)

Captures 32-bit operands arriving from memory or peripherals via the same 8-bit bus—four clock cycles are needed for a full word.

- 00 – sign-extend incoming byte, fill upper 24 bits, write result.
- 01 – zero-extend incoming byte.
- 10 – left-shift existing word by eight, append new byte.
- 11 – right-shift existing word by eight, prepend new byte.

```

module DataRegister(I, E, FunSel, Clock, DROut);
    input wire [7:0] I;
    input wire E;
    input wire [1:0] FunSel;
    input wire Clock;
    output reg [31:0] DROut;

    always@(posedge Clock)
    begin
        if(E)
        begin
            case(FunSel)
                2'b00 : DROut <= {{24{I[7]}}, I}; // 8 bit sign extension
                2'b01 : DROut <= {24'b0, I};
                2'b10 : DROut <= {DROut[23:0], I};
                2'b11 : DROut <= {I, DROut[31:8]};
            endcase
        end
    end
endmodule

```

Instead of shifting we have used concatenations.

2.5 2.4 — General and Scratch Register File (RF)

We have used multiplexer approach. Two independent 3-bit selectors (**OutASel**, **OutBSel**) drive a pair of multiplexers so that **any** desired register's data can be selected to the output bus. **OutA** and **OutB**.

- **RegSel**[3:0] chooses which of the user registers (R-bank) are updated this cycle.
- **ScrSel**[3:0] performs the same role for the scratch bank.
- The 3-bit **FunSel** codes reuse the action list introduced in Section 2.1, plus three byte/half-word helpers — total of eight possibilities.

```
module RegisterFile(I, RegSel, ScrSel, FunSel,
    OutASel, OutBSel, Clock, OutA, OutB);
    input wire [31:0] I;
    input wire [3:0] RegSel;
    input wire [3:0] ScrSel;
    input wire [2:0] FunSel;
    input wire [2:0] OutASel;
    input wire [2:0] OutBSel;
    input wire Clock;
    output reg [31:0] OutA;
    output reg [31:0] OutB;
    wire [31:0] ROut [0:3];
    Register32bit R1(.I(I), .E(RegSel[3]), .FunSel(FunSel), .Clock(Clock), .Q(ROut[0]));
    Register32bit R2(.I(I), .E(RegSel[2]), .FunSel(FunSel), .Clock(Clock), .Q(ROut[1]));
    Register32bit R3(.I(I), .E(RegSel[1]), .FunSel(FunSel), .Clock(Clock), .Q(ROut[2]));
    Register32bit R4(.I(I), .E(RegSel[0]), .FunSel(FunSel), .Clock(Clock), .Q(ROut[3]));
    wire [31:0] SOut [0:3];
    Register32bit S1(.I(I), .E(ScrSel[3]), .FunSel(FunSel), .Clock(Clock), .Q(SOut[0]));
    Register32bit S2(.I(I), .E(ScrSel[2]), .FunSel(FunSel), .Clock(Clock), .Q(SOut[1]));
    Register32bit S3(.I(I), .E(ScrSel[1]), .FunSel(FunSel), .Clock(Clock), .Q(SOut[2]));
    Register32bit S4(.I(I), .E(ScrSel[0]), .FunSel(FunSel), .Clock(Clock), .Q(SOut[3]));

    always@(*) // Multiplexer: connects only one register's output to OutA at a time based on OutASel (3rd State buffer is also an alternative)
    begin
        case (OutASel)
            3'b000 : OutA = ROut[0];
            3'b001 : OutA = ROut[1];
            3'b010 : OutA = ROut[2];
            3'b011 : OutA = ROut[3];
            3'b100 : OutA = SOut[0];
            3'b101 : OutA = SOut[1];
            3'b110 : OutA = SOut[2];
            3'b111 : OutA = SOut[3];
            default: OutA = 32'b0;
        endcase
        case (OutBSel)
            3'b000 : OutB = ROut[0];
            3'b001 : OutB = ROut[1];
            3'b010 : OutB = ROut[2];
            3'b011 : OutB = ROut[3];
            3'b100 : OutB = SOut[0];
            3'b101 : OutB = SOut[1];
            3'b110 : OutB = SOut[2];
            3'b111 : OutB = SOut[3];
            default: OutB = 32'b0;
        endcase
    end
endmodule
```

2.6 2.5 — Address Register File (ARF)

PC (program counter), **SP** (stack pointer), **AR** (auxiliary address), each 16 bits.

Selectors **OutCSel** and **OutDSel** choose any two of the three registers for immediate use on **OutC** and **OutD**.

- **RegSel**[2:0] is a mask; exactly one of the three registers can be targeted per cycle.
- **FunSel**[1:0] selects between decrement, increment, load, and clear—identical semantics to the 16-bit cell in Section 2.1.

```

Register16bit PC(
    .FunSel(FunSel),
    .I(I[15:0]),
    .E(RegSel[2]),
    .Clock(Clock),
    .Q(AROut[1])
);

Register16bit SP(
    .FunSel(FunSel),
    .I(I[15:0]),
    .E(RegSel[1]),
    .Clock(Clock),
    .Q(AROut[2])
);

Register16bit AR(
    .FunSel(FunSel),
    .I(I[15:0]),
    .E(RegSel[0]),
    .Clock(Clock),
    .Q(AROut[3])
);

always@(*)
begin
    case(OutCSel)
        2'b00 : OutC = AROut[1];
        2'b01 : OutC = AROut[2];
        2'b10 : OutC = AROut[3];
        2'b11 : OutC = AROut[3];
    endcase
end

always@(*)
begin
    case(OutDSel)
        2'b00 : OutD = AROut[1];
        2'b01 : OutD = AROut[2];
        2'b10 : OutD = AROut[3];
        2'b11 : OutD = AROut[3];
    endcase
end

endmodule

```

1. Three instances of the generic 16-bit register module (`Register16bit`) are created:

- `PC_reg` writes when `RegSel[2]=1`.
- `SP_reg` writes when `RegSel[1]=1`.
- `AR_reg` writes when `RegSel[0]=1`.

All three share the low half of `I[15:0]` and the same `FunSel`.

2. Two combinational blocks ('always @(*)') implement the read ports: `OutC` and `OutD` each select one of the three internal outputs based on `OutCSel` / `OutDSel`:

- '00' → PC's output
- '01' → SP's output
- otherwise → AR's output

3. Writes are synchronous on the rising edge of `Clock` (only the enabled register updates), while reads change immediately when the select signals change.

2.7 Part 3: Arithmetic Logic Unit

In the third part of the project, we built an Arithmetic Logic Unit (ALU). The ALU has six ports: **A** and **B** are 32-bit inputs, **FunSel** is used as a selector, **Cin** provides the carry input from the Flag Register, and **FlagOut** sets the value of the Flag register when **WF(enable)** is high. **ALU Out** holds the result of the operation performed by the ALU and is 32 bits wide.

We used a 4-bit Flag register where each bit represents an output condition (Zero, Carry, Negative, and Overflow).

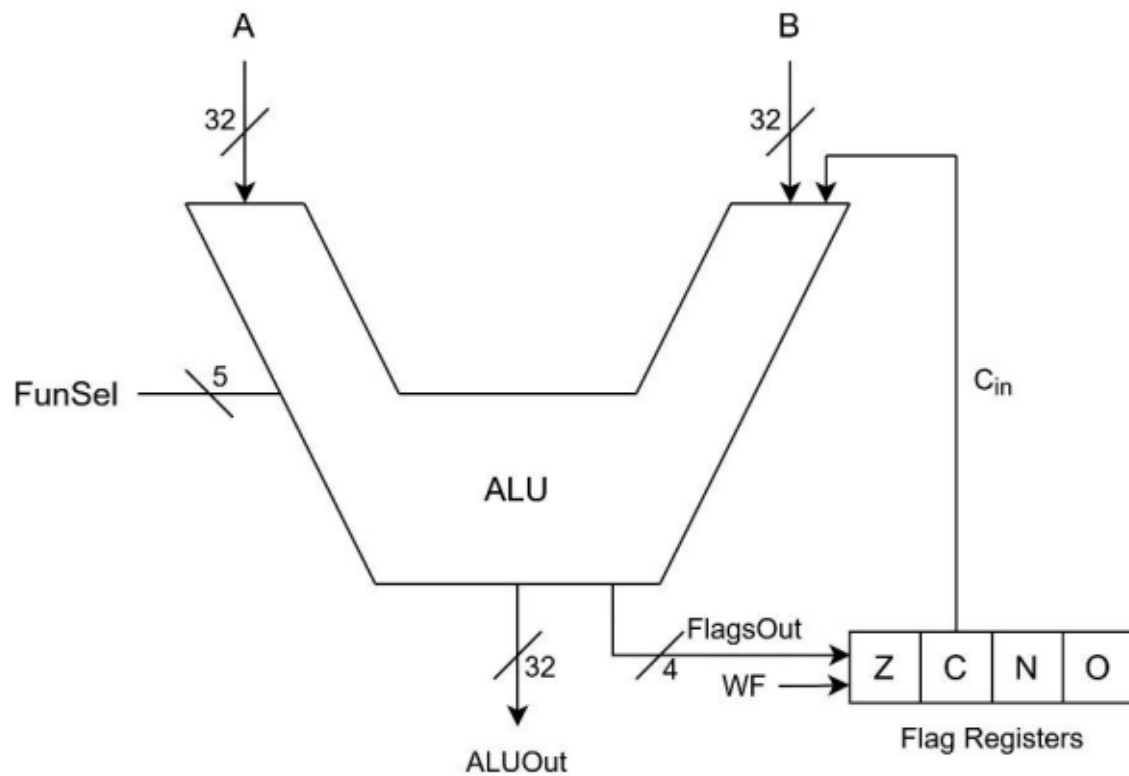


Figure 3: *Block diagram of the Arithmetic Logic Unit.*

FunSel is 5 bits long, allowing our ALU to support 32 unique operations including arithmetic, logic, and shifting. The table below presents the operation encodings.

FunSel	ALUOut	Z	C	N	O
00000	A (8-bit)	+	-	+	-
00001	B (8-bit)	+	-	+	-
00010	NOT A (8-bit)	+	-	+	-
00011	NOT B (8-bit)	+	-	+	-
00100	A + B (8-bit)	+	+	+	+
00101	A + B + Carry (8-bit)	+	+	+	+
00110	A – B (8-bit)	+	+	+	+
00111	A AND B (8-bit)	+	-	+	-
01000	A OR B (8-bit)	+	-	+	-
01001	A XOR B (8-bit)	+	-	+	-
01010	A NAND B (8-bit)	+	-	+	-
01011	LSL A (8-bit)	+	+	+	-
01100	LSR A (8-bit)	+	+	+	-
01101	ASR A (8-bit)	+	+	-	-
01110	CSL A (8-bit)	+	+	+	-
01111	CSR A (8-bit)	+	+	+	-

FunSel	ALUOut	Z	C	N	O
10000	A (16-bit)	+	-	+	-
10001	B (16-bit)	+	-	+	-
10010	NOT A (16-bit)	+	-	+	-
10011	NOT B (16-bit)	+	-	+	-
10100	A + B (16-bit)	+	+	+	+
10101	A + B + Carry (16-bit)	+	+	+	+
10110	A – B (16-bit)	+	+	+	+
10111	A AND B (16-bit)	+	-	+	-
11000	A OR B (16-bit)	+	-	+	-
11001	A XOR B (16-bit)	+	-	+	-
11010	A NAND B (16-bit)	+	-	+	-
11011	LSL A (16-bit)	+	+	+	-
11100	LSR A (16-bit)	+	+	+	-
11101	ASR A (16-bit)	+	+	-	-
11110	CSL A (16-bit)	+	+	+	-
11111	CSR A (16-bit)	+	+	+	-

Figure 4: *ALU operation encoding table.*

Some arithmetic operations operate on 16-bit slices of the inputs, which requires sign extension for correct operation. Shift operations store a bit in the carry flag, as illustrated below.

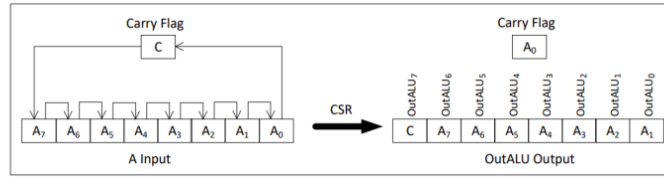


Figure 5: *Circular shift operation.*

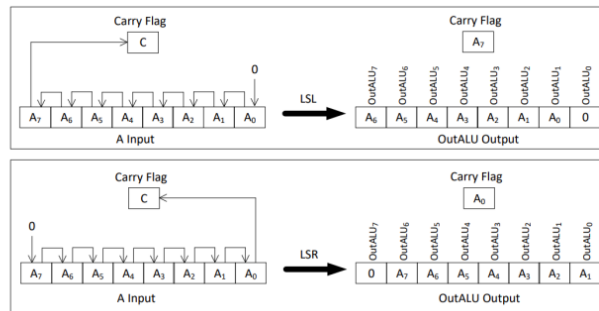


Figure 6: *Logical shift operation.*

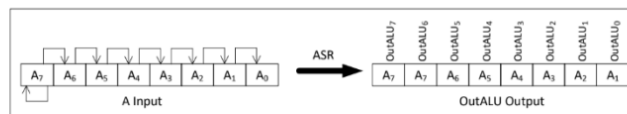


Figure 7: *Arithmetic shift.*

2.8 Part 4: Arithmetic Logic Unit System

In this part, all the modules built in the previous parts were combined to construct a simplified CPU. The only new module in this part was Multiplexer (MUX), a combination circuit that selects a value from multiple inputs depending on the selection value given. In this project, MUX 4 to 1 and MUX 2 to 1 were used. Moreover, we used memory, which contains five ports: Address port specifies the address that data is going to read from or written on, data port carries the data that will be written on memory, WR (Write/Read) control signal specifies whether the memory will be accessed for reading or writing, The CS (Chip Select) signal enables the memory, MemOut carries the data output from the memory, and the memory module also includes a clock port for synchronization.

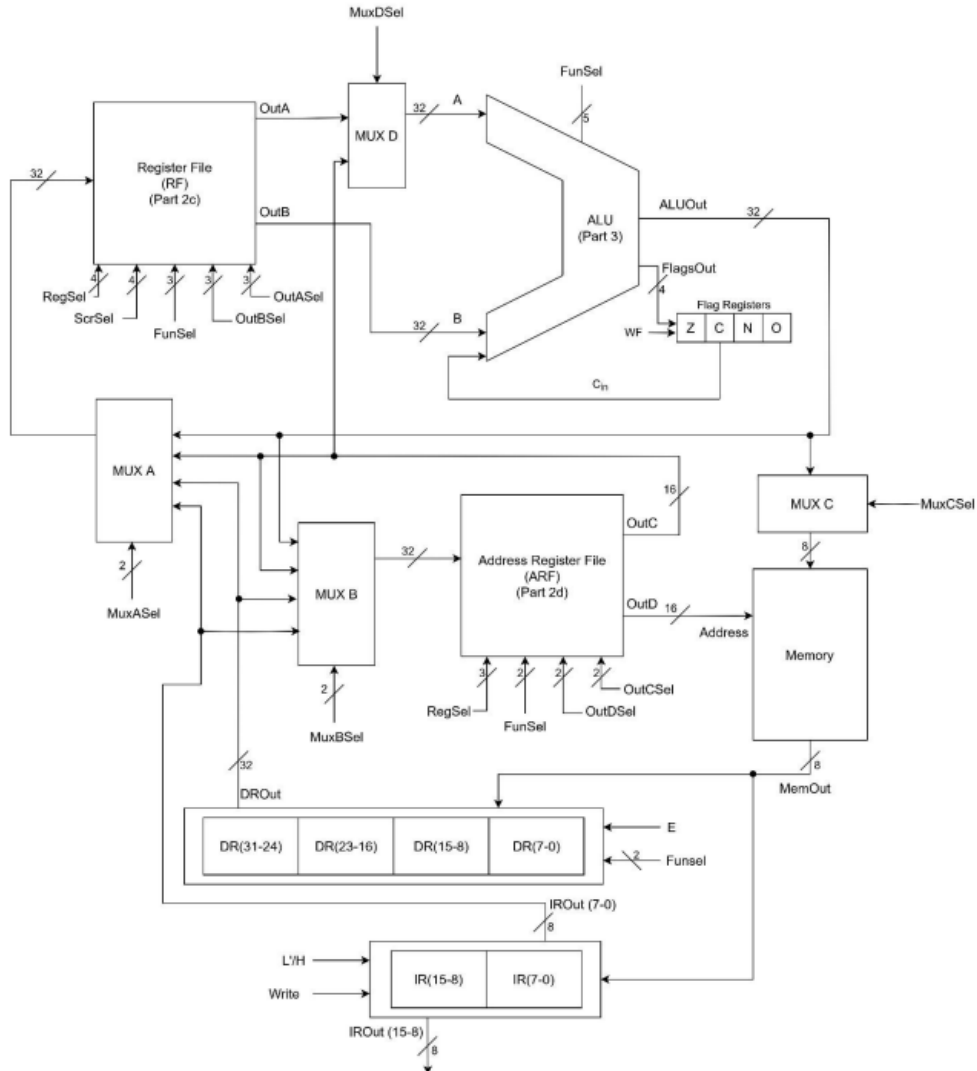


Figure 8: *Arithmetic shift.*

3 RESULTS

Following the implementation of each component, we ran the relevant behavioral simulation files and looked at the results. Every test case has been successfully completed. Following that, we also made sure our project could run.bat and pass all of the given scenarios.

In part 1-a of the project, we were asked to implement a 16-bit register. This register will be running according to the value carried in FunSel port. Figure 9 shows the result of the simulation file for the 16-bit register and the different operations depending upon FunSel values.

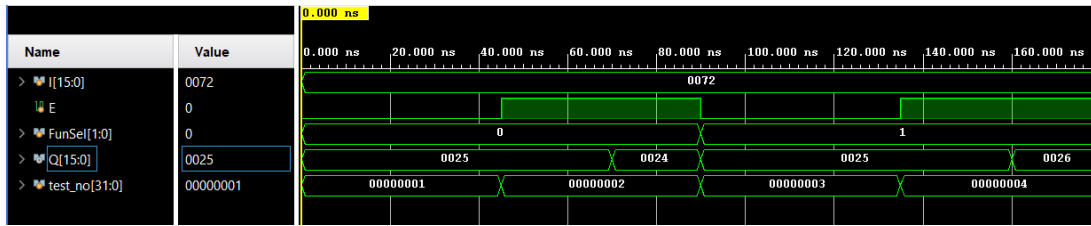


Figure 9: *Register16bitSimulation.v Simulation*

In part 1-b, we implemented a 32-bit register. Our model will execute eight different functions depending on the value of FunSel. Figure 10 shows the result of running the simulation file of this model.

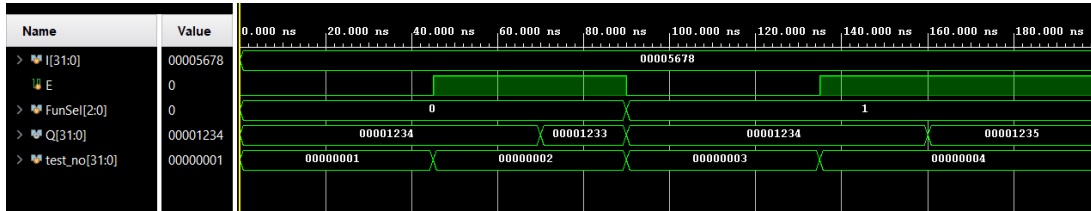


Figure 10: *Register36bitSimulation.v Simulation*

In part 2-a we were asked to implement a 16-bits Instruction Register (IR). The IR has eight-bit input ports, and it loads the input value into either the lower (bits 7-0) or higher (bits 15-8) depending on the value of the L'H controller. Figure 11 shows the result of running the Instruction Register Simulation file.

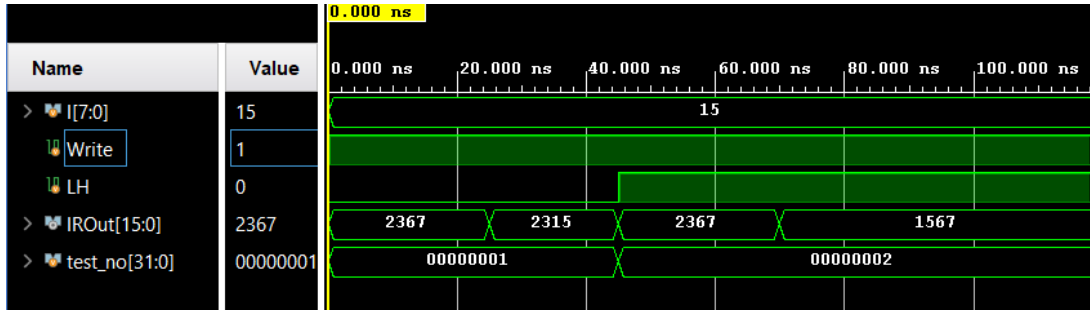


Figure 11: *InstructionRegisterSimulation.v Simulation*

In part 2-b of the project, we built a 32-bit Data Register (DR). The DR register file's input data bus has an 8-bit limit. As a result, the register can only hold 8 bits of data each clock cycle, meaning it will take four cycles to load all 32 bits of data. The DR has a bits FunSel controller, which allows it to execute four different functions. The execution of the DR Simulation file is shown in Figure 12.

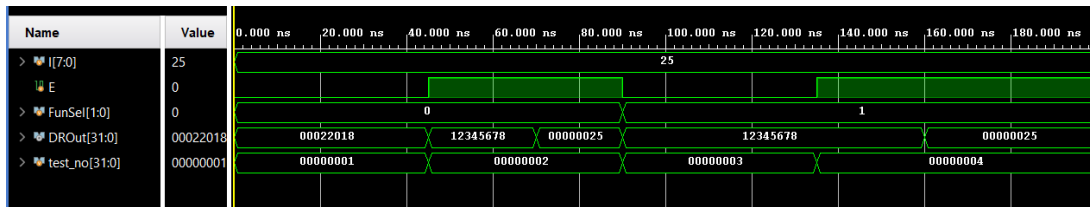


Figure 12: *DataRegisterSimulation.v Simulation*

The project's step 2-c involved building the Register File (RF) model. Six 32-bit Register models were utilized to implement RF; they were implemented in part 1-b. Of the 32 registers in RF, three were utilized for general-purpose purposes and three as scratch registers. As a function depends on the inputs it gets, Figure 13 below displays the outcome of running the Register File simulation.

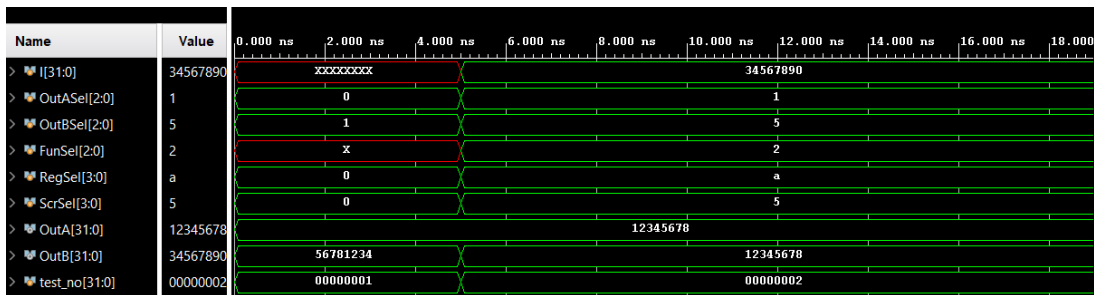


Figure 13: *RegisterFileSimulation.v Simulation*

In part 2-d of this project Address Register File (ARF) model was designed. This model included three 16-bit registers implemented in part 1-b they were used as program

indicating that our models progressed correctly. The schematic Figure 16 and simulation results in Figure 9 also validate our implementation.

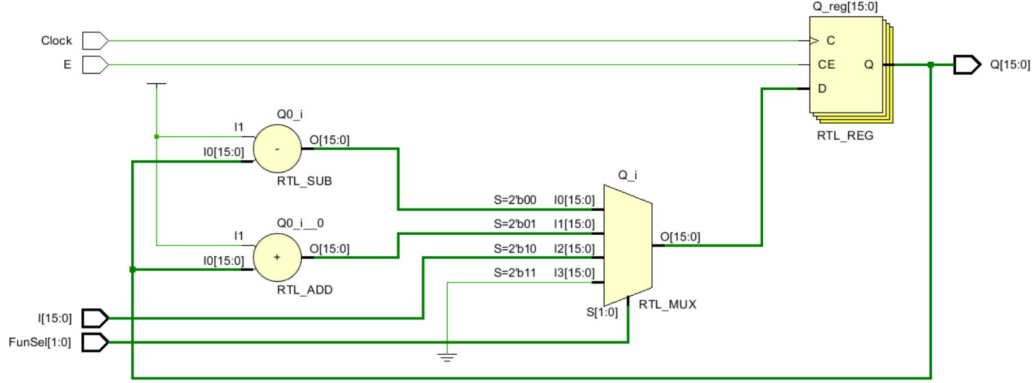


Figure 16: 16 Bit Register Schematic

Part b was almost similar to Part A, we created a 32-Bit Register model instead of a 16-bit Register. Besides that this model can execute eight functions instead of only 4 in part A. We ran the test file to check our model and how it executes functions and it succeeded in passing all the tests. Moreover, the schematic in Figure 17 design of the model provides clear evidence that it has been correctly and thoughtfully constructed.

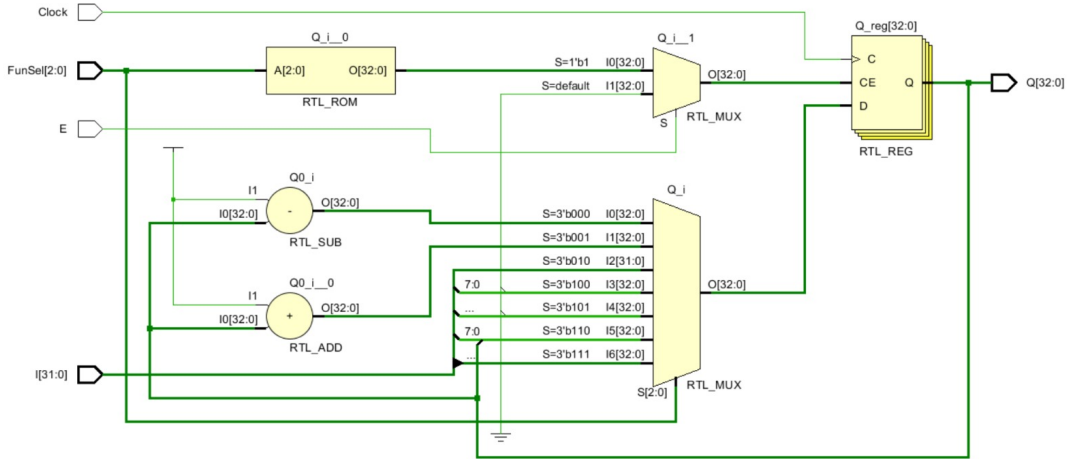


Figure 17: 32 Bit Register Schematic

Part 2 consists of 4 parts: a, b, c and d. We created a 16-bit instruction register model that, based on the L'H input value, determines whether to load an 8-bit input in the lower or upper half of the register. Following this approach, we evaluated the model using the test file that was supplied, and it passed every test, proving that it functions as

intended. Furthermore, Figure 18's schematic representation of this model clearly shows that this function operates as intended.

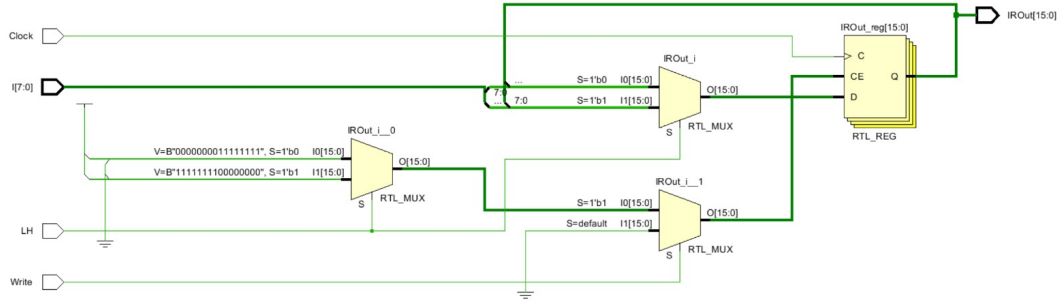


Figure 18: *Instruction Register Schematic*

In part b, We designed a 32-bit Data Register (DR) model capable of receiving 8-bit input and intelligently selecting the correct segment of the register for data storage and what operation to operate depending on the funnel value. After this design, we tested the model using the provided test file and it passed all the tests indicating that it works correctly. Moreover, the schematic design of this model in Figure 19 strongly demonstrates that this function works correctly.

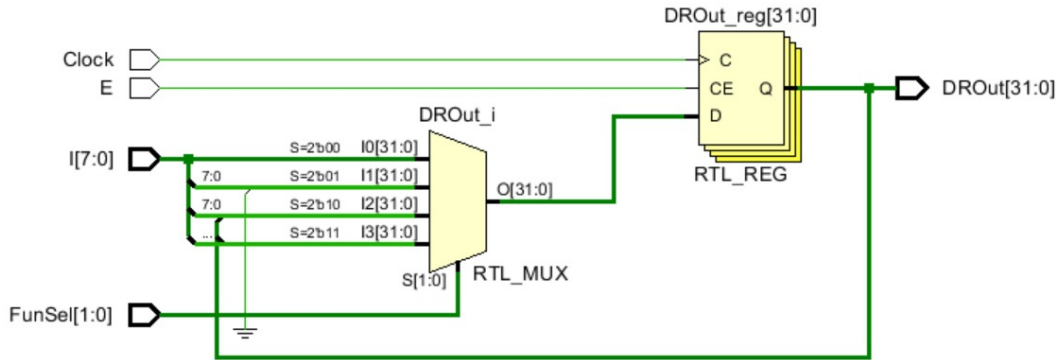


Figure 19: *Data Register Schematic*

In part c, we used the six 32bit Register model created in part one instead of building the 32bit Register from scratch. Three of these registers were used as general-purpose registers and three were used as scratch registers. Also, this model was unique because it has two different outputs and each output value depended on its own OutSel value. Implementing this model was a bit complicated, however, we successfully implemented it

and it passed all the provided tests. We also operated the synthesizer to get the schematic design that is displayed in Figure 20. We implemented

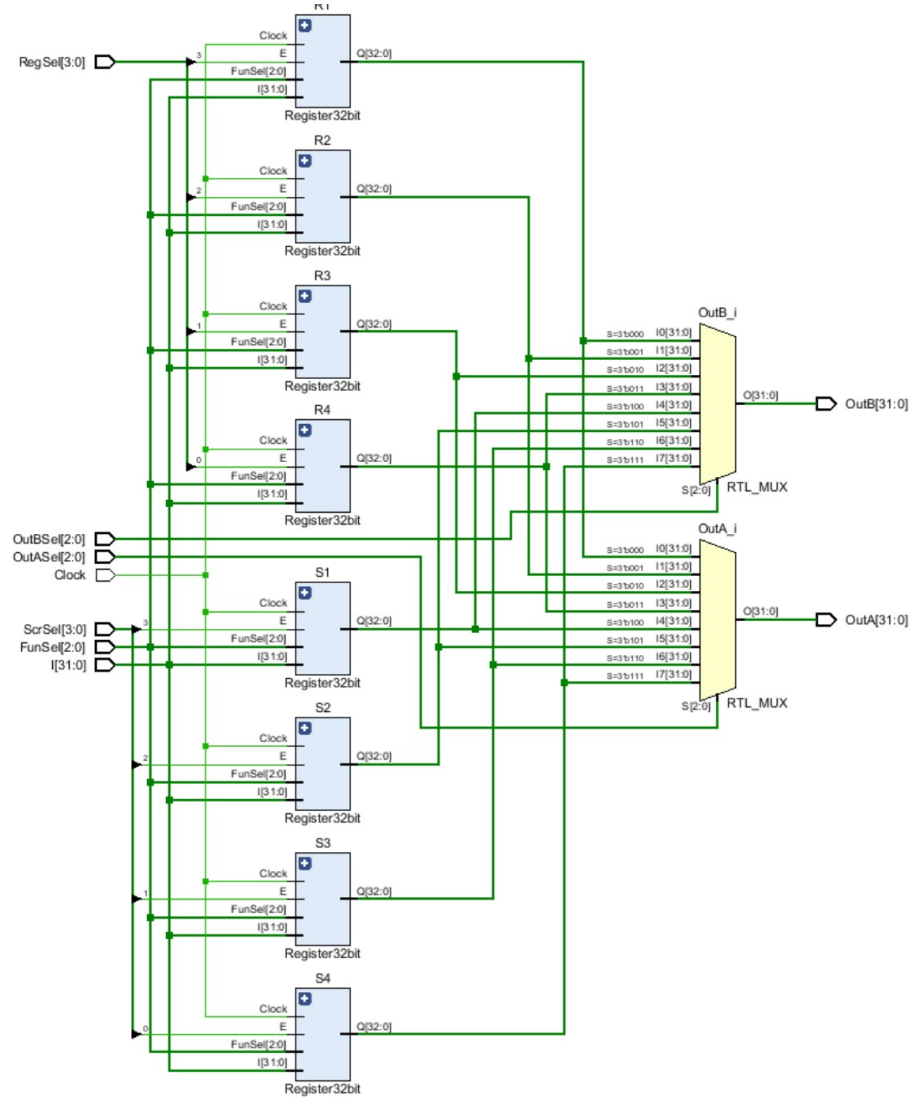


Figure 20: *Register File Schematic*

Part d was similar to part c but simpler. We were asked to implement an Address Register File consisting of three 16-bit Registers: program counter (PC), address register (AR), and stack pointer (SP). It also has two outputs with 16-bit length and its output selection mechanism is almost similar to part c however OutSel is two bits long instead of three. Figure 21 shows the schematic design of the Address Register File.

passed all test cases. Figure 23 depicts the schematic design of our ALU system model.

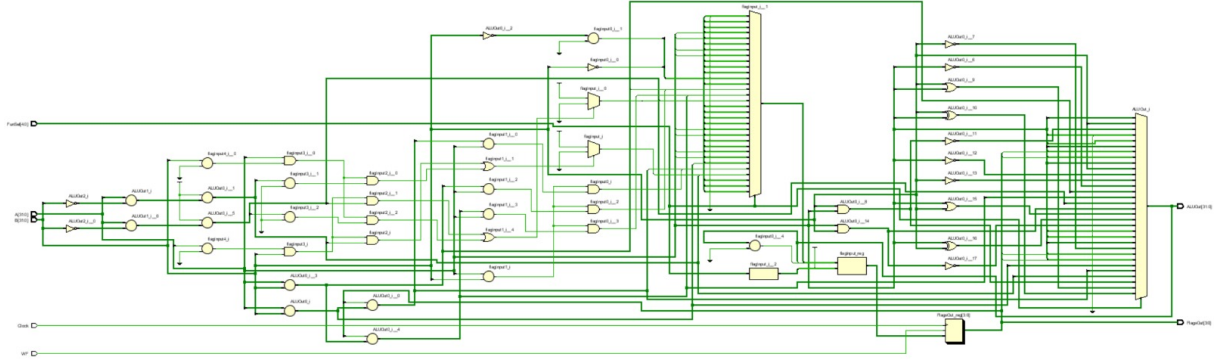


Figure 23: *Arithmetic Logic Unit Schematic*

5 CONCLUSION

We faced some difficulties in this project because it was the first time we used Verilog to implement hardware design. This project required time arrangement and teamwork skills because it consisted of 4 parts which consumed a lot of time. Despite the difficulties we faced in this project, Doing the project helped us discover the Verilog programming language. Moreover, it helped us study for this class and see the implementation of what we learned in the class in simulation improving and deepening our understanding of computer organization.